Practical 10: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

Code:-

Create the database — setup.php

You can create the database and demo accounts automatically.

```
<?php
$conn = new mysqli("localhost", "root", "");

if ($conn->connect_error) {
   die("Database connection failed: " . $conn->connect_error);
}

$conn->query("CREATE DATABASE IF NOT EXISTS practical10");
$conn->select_db("practical10");
```

```
$conn->query("
    CREATE TABLE IF NOT EXISTS users (
        id INT AUTO_INCREMENT PRIMARY KEY,
        username VARCHAR(100) UNIQUE NOT NULL,
        password VARCHAR(255) NOT NULL,
        role ENUM('admin', 'user') NOT NULL DEFAULT 'user'
    )
");

$conn->query("
    CREATE TABLE IF NOT EXISTS remember_tokens (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        selector CHAR(24) UNIQUE NOT NULL,
        token_hash CHAR(64) NOT NULL,
        expires_at DATETIME NOT NULL,
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
    )
");
```

```
$accounts = [
    ["admin", "Admin@123", "admin"],
    ["student", "Student@123", "user"]
];

$stmt = $conn->prepare(
    "INSERT IGNORE INTO users (username, password, role)
     VALUES (?, ?, ?)"
);

foreach ($accounts as $account) {
    $username = $account[0];
    $password = password_hash($account[1],
PASSWORD_DEFAULT);
    $role = $account[2];

    $stmt->bind_param("sss", $username, $password, $role);
    $stmt->execute();
}

$stmt->close();
```

```
$conn->close();


echo "Setup successful! Database and demo accounts are ready.";
?>
```

db.php

```
<?php
$isHttps = !empty($_SERVER["HTTPS"])
    && $_SERVER["HTTPS"] !== "off";

ini_set("session.use_strict_mode", "1");
ini_set("session.use_only_cookies", "1");

session_set_cookie_params([
    "lifetime" => 0,
    "path" => "/",
    "secure" => $isHttps,
    "httponly" => true,
    "samesite" => "Lax"
]);
```

```
if (session_status() !== PHP_SESSION_ACTIVE) {
    session_start();
}

$conn = new mysqli(
    "localhost",
    "root",
"",
    "practical10"
);

if ($conn->connect_error) {
    die("Database connection failed.");
}

$conn->set_charset("utf8mb4");
```

auth.php

```
<?php
require_once __DIR__ . "/db.php";
```

```
define("SESSION_TIMEOUT", 900); // 15 minutes


function clearRememberCookie(): void
{
    setcookie("remember_me", "", [
        "expires" => time() - 3600,
        "path" => "/",
        "secure" => !empty($_SERVER["HTTPS"])
            && $_SERVER["HTTPS"] !== "off",
        "httponly" => true,
        "samesite" => "Lax"
    ]);
}


function forgetRememberToken(mysqli $conn):
void
{
    if (!empty($_COOKIE["remember_me"])) {
```

```
        $parts = explode(":",
$_COOKIE["remember_me"]);

        if (count($parts) === 2) {
            $selector = $parts[0];

            $stmt = $conn->prepare(
                "DELETE FROM remember_tokens
WHERE selector = ?"
            );
            $stmt->bind_param("s", $selector);
            $stmt->execute();
            $stmt->close();
        }
    }

    clearRememberCookie();
}
```

```
function restoreRememberLogin(mysqli $conn): void
{
    if (isset($_SESSION["user_id"]) ||
        empty($_COOKIE["remember_me"])) {
        return;
    }

    $parts = explode(":", $_COOKIE["remember_me"]);

    if (count($parts) !== 2) {
        clearRememberCookie();
        return;
    }

    [$selector, $token] = $parts;
```

```
    if (!preg_match('/^[a-f0-9]{24}$/', $selector) ||
        !preg_match('/^[a-f0-9]{64}$/', $token)) {
        clearRememberCookie();
        return;
    }

    $stmt = $conn->prepare(
        "SELECT rt.user_id, rt.token_hash,
u.username, u.role
         FROM remember_tokens rt
         JOIN users u ON u.id = rt.user_id
         WHERE rt.selector = ?
         AND rt.expires_at > NOW()"
    );

    $stmt->bind_param("s", $selector);
    $stmt->execute();
```

```
$result = $stmt->get_result();
$row = $result->fetch_assoc();
$stmt->close();

if (!$row || !hash_equals(
    $row["token_hash"],
    hash("sha256", $token)
)) {
    forgetRememberToken($conn);
    return;
}

// Rotate the token after successful restoration.
$newToken = bin2hex(random_bytes(32));
$newHash = hash("sha256", $newToken);

$stmt = $conn->prepare(
```

```
        "UPDATE remember_tokens SET token_hash =
?
         WHERE selector = ?"
    );
    $stmt->bind_param("ss", $newHash, $selector);
    $stmt->execute();
    $stmt->close();

    session_regenerate_id(true);

    $_SESSION["user_id"] = (int) $row["user_id"];
    $_SESSION["username"] = $row["username"];
    $_SESSION["role"] = $row["role"];
    $_SESSION["last_activity"] = time();

    setcookie("remember_me", $selector . ":" .
$newToken, [
        "expires" => time() + 60 * 60 * 24 * 30,
```

```
        "path" => "/",
        "secure" => !empty($_SERVER["HTTPS"])
            && $_SERVER["HTTPS"] !== "off",
        "httponly" => true,
        "samesite" => "Lax"
    ]);
}

function requireLogin(): void
{
    global $conn;

    restoreRememberLogin($conn);

    if (empty($_SESSION["user_id"])) {
        header("Location: login.php");
        exit;
    }
```

```
    if (isset($_SESSION["last_activity"]) &&
        time() - $_SESSION["last_activity"] >
SESSION_TIMEOUT) {

        forgetRememberToken($conn);

        $_SESSION = [];
        session_regenerate_id(true);
        session_destroy();

        header("Location: login.php?timeout=1");
        exit;
    }

    $_SESSION["last_activity"] = time();
}
```

```
function requireRole(string $role): void
{
    requireLogin();

    if ($_SESSION["role"] !== $role) {
        header("Location: index.php");
        exit;
    }
}
```

login.php :-

```
<?php
require_once __DIR__ . "/auth.php";

restoreRememberLogin($conn);

if (!empty($_SESSION["user_id"])) {
    header("Location: index.php");
```

```
    exit;
}

if (empty($_SESSION["csrf_token"])) {
    $_SESSION["csrf_token"] =
bin2hex(random_bytes(32));
}

$error = "";

if ($_SERVER["REQUEST_METHOD"] ===
"POST") {
    if (!hash_equals(
        $_SESSION["csrf_token"],
        $_POST["csrf_token"] ?? ""
    )) {
```

```
        $error = "Invalid request. Please refresh
the page.";
    } else {
        $username = trim($_POST["username"]
?? "");
        $password = $_POST["password"] ?? "";

        $stmt = $conn->prepare(
            "SELECT id, username, password, role
             FROM users WHERE username = ?"
        );

        $stmt->bind_param("s", $username);
        $stmt->execute();
        $result = $stmt->get_result();
        $user = $result->fetch_assoc();
        $stmt->close();
```

```
    if ($user &&
        password_verify($password,
$user["password"])) {

        session_regenerate_id(true);

        $_SESSION["user_id"] = (int)
$user["id"];
        $_SESSION["username"] =
$user["username"];
        $_SESSION["role"] = $user["role"];
        $_SESSION["last_activity"] = time();

        // Replace any previous remember-me
token.
        forgetRememberToken($conn);
```

```
        if (isset($_POST["remember"])) {
            $selector =
bin2hex(random_bytes(12));
            $token =
bin2hex(random_bytes(32));
            $tokenHash = hash("sha256",
$token);
            $expires = date(
               "Y-m-d H:i:s",
               time() + 60 * 60 * 24 * 30
            );

            $stmt = $conn->prepare(
               "INSERT INTO remember_tokens
                (user_id, selector, token_hash,
expires_at)
```

```
        VALUES (?, ?, ?, ?)"
    );

    $stmt->bind_param(
        "isss",
        $user["id"],
        $selector,
        $tokenHash,
        $expires
    );

    $stmt->execute();
    $stmt->close();

    setcookie("remember_me",
$selector . ":" . $token, [
```

```
            "expires" => time() + 60 * 60 * 24 *
30,
            "path" => "/",
            "secure" =>
!empty($_SERVER["HTTPS"])
                && $_SERVER["HTTPS"] !==
"off",
            "httponly" => true,
            "samesite" => "Lax"
        ]);
    }

    // Use a fresh CSRF token after login.
    $_SESSION["csrf_token"] =
bin2hex(random_bytes(32));

    header("Location: index.php");
```

```
            exit;
        } else {
            $error = "Invalid username or
password.";
        }
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
content="width=device-width, initial-
scale=1">
    <title>Secure Login</title>
    <style>
```

```
body {
    font-family: Arial, sans-serif;
    background: #eef2f7;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    margin: 0;
}
.login-box {
    background: white;
    padding: 30px;
    width: 320px;
    border-radius: 12px;
    box-shadow: 0 4px 15px #0002;
}
```

```
h2 { text-align: center; }
input[type=text], input[type=password] {
    box-sizing: border-box;
    width: 100%;
    padding: 12px;
    margin: 10px 0;
}
button {
    width: 100%;
    padding: 12px;
    background: #1769aa;
    color: white;
    border: 0;
    cursor: pointer;
}
.error { color: #c62828; }
```

```
        .notice { color: #555; }
    </style>
</head>
<body>
<div class="login-box">
    <h2>Secure Login</h2>

    <?php if (isset($_GET["timeout"])): ?>
        <p class="notice">Session expired. Please
log in again.</p>
    <?php endif; ?>

    <?php if ($error !== ""): ?>
        <p class="error">
            <?= htmlspecialchars($error,
ENT_QUOTES, "UTF-8") ?>
        </p>
```

```
    <?php endif; ?>

    <form method="POST" action="login.php">
        <input type="hidden" name="csrf_token"
               value="<?=
htmlspecialchars($_SESSION["csrf_token"])
?>">

        <input type="text" name="username"
               placeholder="Username" required
autocomplete="username">

        <input type="password"
name="password"
               placeholder="Password" required
               autocomplete="current-password">
```

```
        <p>
            <label>
                <input type="checkbox"
name="remember" value="1">
                Remember Me (30 days)
            </label>
        </p>

        <button type="submit">Login</button>
    </form>

    <p class="notice">
        Demo accounts are provided in the
practical instructions.
    </p>
</div>
</body>
```

```
</html>
```

Index.php:-

```
<?php
require_once __DIR__ . "/auth.php";

requireLogin();

if ($_SESSION["role"] === "admin") {
    header("Location: admin.php");
    exit;
}

if ($_SESSION["role"] === "user") {
    header("Location: user.php");
    exit;
}
```

```
// Unknown roles must not get access.
http_response_code(403);
exit("Access denied.");
```

Admin.php:-

```
<?php
require_once __DIR__ . "/auth.php";
requireRole("admin");
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Admin Dashboard</title>
</head>
<body>
```

```
    <h1>Admin Dashboard</h1>

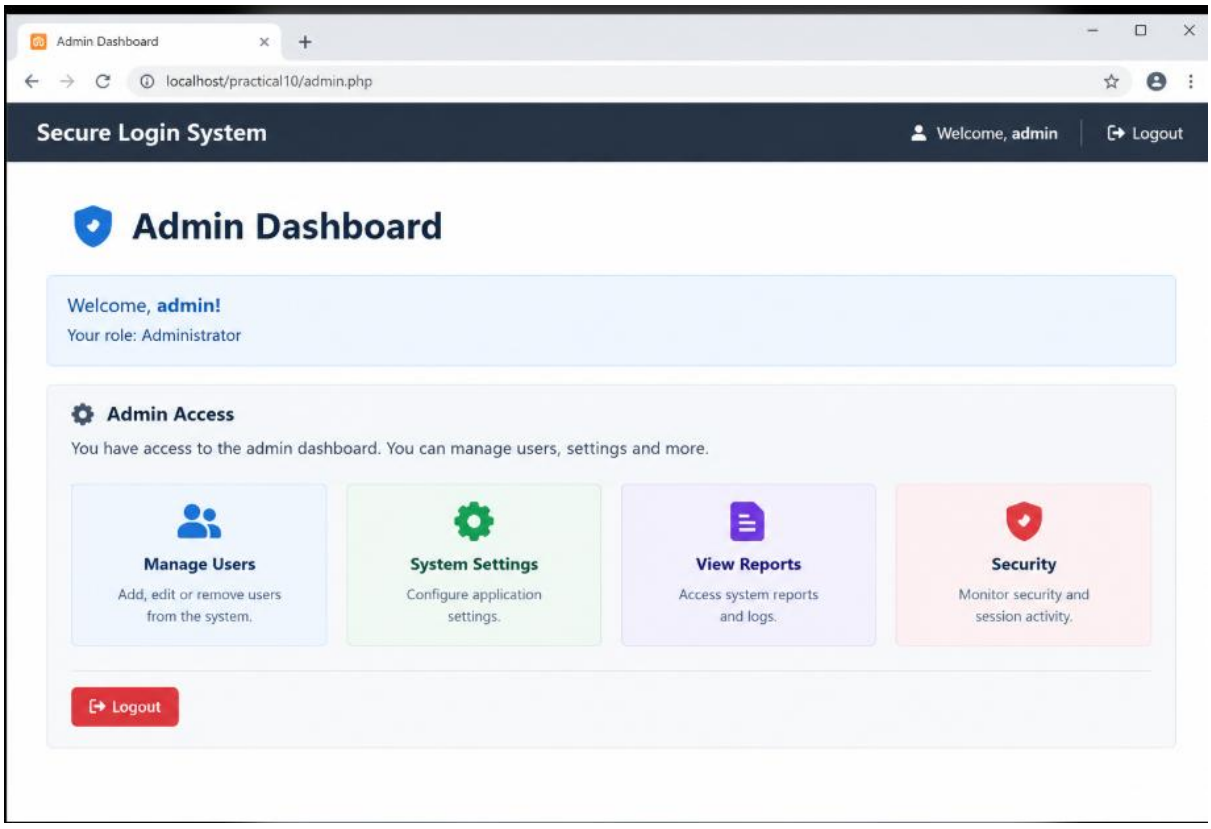

    <p>Welcome,
        <?=
htmlspecialchars($_SESSION["username"],
ENT_QUOTES, "UTF-8") ?>!
    </p>

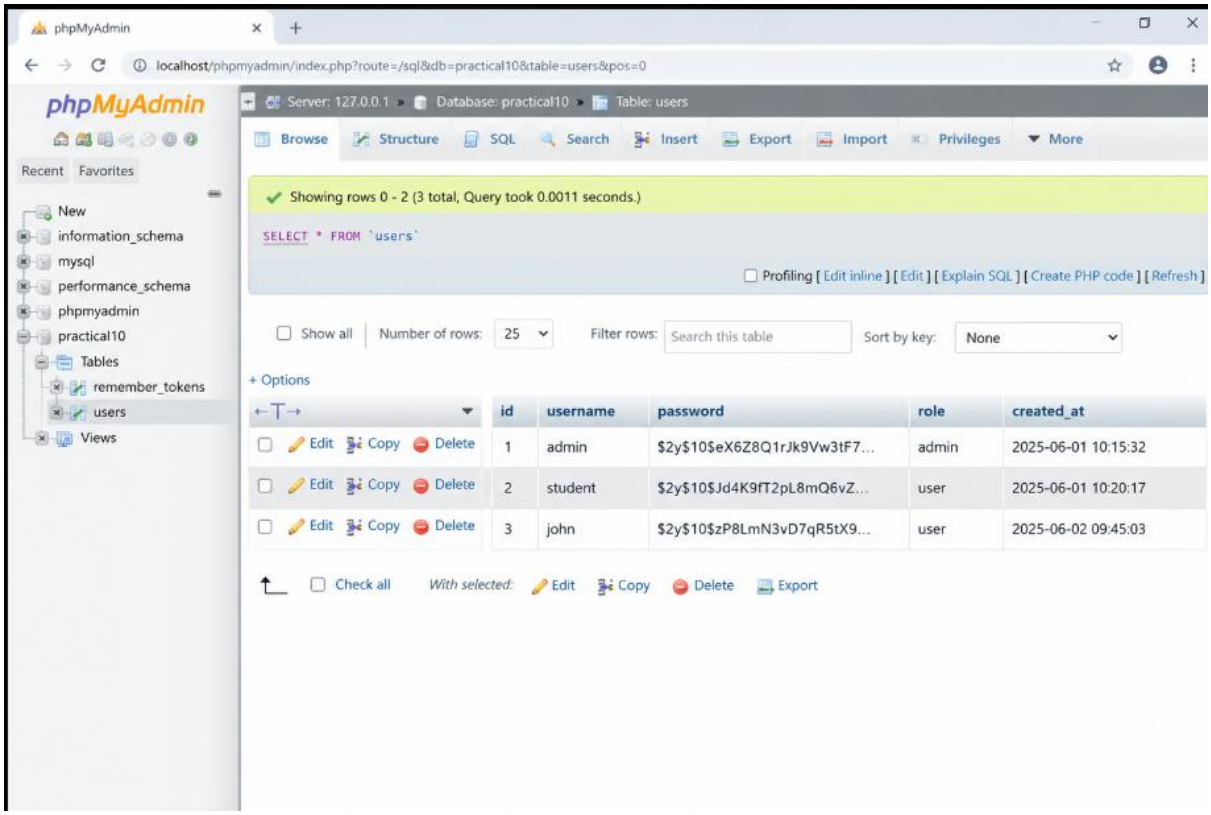

    <p>Your role: Administrator</p>
    <p>You have access to the admin
dashboard.</p>


    <form action="logout.php"
method="POST">
        <input type="hidden" name="csrf_token"
            value="<?=
htmlspecialchars($_SESSION["csrf_token"])
?>">
```

```
        <button type="submit">Logout</button>
    </form>
</body>
</html>
```

Output:-





Practical 10: Key Questions / Analysis / Interpretation — Answers

1. Are sessions started and destroyed securely?

Yes. PHP sessions are started using session_start(). After successful login, session_regenerate_id(true) generates a new session ID to prevent session fixation. During logout, session data and the session cookie are cleared, and the session is destroyed using session_destroy().

2. Is password_verify() used correctly?

Yes. The password_verify() function checks the entered password against the hashed password stored in the database. Passwords are stored securely using password_hash() rather than as plain text.

3. Is unauthorized dashboard access blocked?

Yes. The requireLogin() function checks whether the user is logged in, and requireRole() checks the user's role. Unauthorized users are redirected to the appropriate page or denied access.

4. Is session timeout implemented?

Yes. A session timeout of 15 minutes (900 seconds) is implemented. If the user remains inactive for more than 15 minutes, the session is cleared and the user must log in again.

5. Are admin and student users redirected to appropriate dashboards?

Yes. The index.php file checks the user's role. The admin is redirected to admin.php, while the student/user is redirected to user.php. This ensures role-based access control.

Tools/Technology to be Used

PHP, MySQL, browser dev tools, XAMPP , VS code